Deep Reinforcement Learning for Aircraft Pulse Assembly Line Scheduling via Heterogeneous Graph and Autoregressive Pointer Networks

Xiao Lu
College of Electronics and Information Engineering
Tongji University
Shanghai, China
luxiao@tongji.edu.cn

Jiakang Ai
College of Electronics and Information Engineering
Tongji University
Shanghai, China
aijiakang@tongji.edu.cn

Fei Qiao
College of Electronics and Information Engineering
Tongji University
Shanghai, China
fqiao@tongji.edu.cn

Juan Liu*
College of Electronics and Information Engineering
Tongji University
Shanghai, China
juanliu0720@tongji.edu.cn

Lingtao Chen
College of Electronics and Information Engineering
Tongji University
Shanghai, China
chenlingtao@tongji.edu.cn

Yumin Ma
College of Electronics and Information Engineering
Tongji University
Shanghai, China
ymma@tongji.edu.cn

***Abstract*—The scheduling of aircraft pulse assembly lines (APALs) represents a critical challenge due to intricate spatial-temporal dependencies and rigid multi-resource constraints involving operations, stations, and skilled workers. This paper proposes an APAL-oriented Heterogeneous Graph Pointer-Proximal Policy Optimization (HGP-PPO) framework for this coupled multi-resource scheduling problem. The scheduling state is represented as a tripartite heterogeneous graph over operations, stations, and workers, and an autoregressive pointer decoder sequentially selects the operation, station, and variable-length worker team, thereby dynamically allocating eligible workers to stations at decision time under adaptive feasibility masks. This design reduces the burden of flattened joint-action modeling while preserving the main APAL feasibility constraints during policy inference. Experiments on four APAL instance scales indicate that HGP-PPO improves over flattened PPO/DQN baselines and remains competitive with the tested heuristic and search-based baselines on larger cases. We further report offline training cost and online inference time to clarify the practical efficiency trade-off of the learned scheduler.**

***Index Terms*—Deep Reinforcement Learning, Aircraft Pulse Assembly Lines, Heterogeneous Graph Neural Network, Pointer Network, Multi-resource Scheduling**

I. Introduction

Aircraft Pulse Assembly Lines (APALs) require a schedule that assigns precedence-constrained operations to physical stations and compatible multi-skilled workers while aircraft sections move synchronously along the line. This problem is challenging because the scheduler must respect temporal precedence, station capacity, pulse-line spatial flow, worker skill compatibility, and station-worker binding at the same time. A practical scheduler should therefore reduce the pulse-cycle-related makespan while generating feasible decisions quickly enough for production planning and adjustment.

This work was supported by the National Natural Science Foundation of China (Nos. 62133011, 62373288, 62403365, 62273260), and the Open Research Project of the State Key Laboratory of Industrial Control Technology of China (No. ICT2025B28). *Corresponding author

Traditional scheduling methods, including mathematical formulations, dispatching rules, and iterative search, have long supported assembly-line balancing and scheduling [1]. They remain useful as interpretable baselines, but their search cost and budget sensitivity become restrictive for APAL instances with thousands of operations and coupled station-worker constraints. Deep Reinforcement Learning (DRL) offers fast scheduling decisions in production systems [2], and Graph Neural Networks (GNNs) have represented disjunctive or heterogeneous scheduling graphs in JSP, distributed JSP, multi-task allocation, and flexible job-shop variants [3]–[10].

Despite this progress, most learning-based schedulers still focus on operation-machine assignment, single-worker allocation, or flattened joint actions. A closely related APAL study by Ai et al. [11] formulates the Type-2 APAL balancing problem with a disjunctive graph and a GNN-based Actor-Critic framework for cycle-time minimization. In contrast, this paper studies scheduling with explicit operation-station-worker-team coupling, decision-time worker-to-station allocation, station-worker binding, and variable-length team construction. Directly flattening this joint action space obscures heterogeneous resource relations and complicates feasibility control, especially when multiple workers must be selected jointly. Table I positions this work against representative learning-based scheduling and allocation methods.

To address this APAL-specific challenge, we propose a Heterogeneous Graph Pointer-Proximal Policy Optimization (HGP-PPO) framework. HGP-PPO represents operations, stations, and workers as a tripartite heterogeneous graph to encode precedence, spatial compatibility, skill matching, and station-worker binding. It further uses an autoregressive pointer decoder with adaptive masks to factorize each joint decision into operation, station, and worker-team selections, reducing the flattened action-space burden while embedding major feasibility constraints.

The main contributions of this paper are summarized as

TABLE I
POSITIONING OF HGP-PPO AGAINST REPRESENTATIVE LEARNING-BASED SCHEDULING AND ALLOCATION METHODS.

Work	Problem setting	Learning/graph model	Resource scope	APAL-specific positioning
Song et al. [3]	Flexible job-shop scheduling	GNN with DRL	Operation-machine scheduling	No pulse-line station binding or worker-team formation
Liu and Huang [4]	Dynamic job-shop scheduling	GNN with DRL	Dynamic scheduling decisions	No APAL spatial-flow or multi-worker team constraints
Huang et al. [5]	Distributed job-shop scheduling	End-to-end GNN-based DRL	Distributed shop-floor scheduling	No APAL operation-station-worker coupling
Zhao et al. [6]	Dependency-aware spatial crowdsourcing	Heterogeneous graph RL	Multi-task allocation	Related graph-RL allocation, but not APAL scheduling
Hu et al. [7]	Dual-resource constrained FJSP	Semantic-aware graph DRL	Dual-resource scheduling	No pulse-line station binding with variable-size worker teams
Zhong et al. [8]	FJSP with skilled workers	Multi-domain graph attention	Skilled-worker-aware scheduling	No APAL operation-station-worker-team decoding
Yi et al. [9]	Dynamic FJSP with multi-skilled workers	End-to-end DRL	Multi-skilled worker constraints	No APAL station binding or pulse-line spatial flow
Yuan et al. [10]	Flexible job-shop scheduling	DRL	Flexible job-shop decisions	No heterogeneous APAL station-worker-team masks
Ai et al. [11]	Type-2 APAL balancing	Disjunctive graph with GNN Actor-Critic	Task-to-workstation assignment with worker types	Closest APAL graph-RL work, but not variable worker-team scheduling with station binding
HGP-PPO	**APAL scheduling**	**Heterogeneous graph pointer PPO**	**Operation-station-variable worker team**	**Targets APAL binding, spatial flow, skills, and variable teams**

follows:

- We formulate APAL scheduling as an event-driven Markov Decision Process with coupled operation-station-worker decisions, explicitly considering pulse-line spatial monotonicity, station capacity, skill compatibility, station-worker binding, and variable-length worker-team formation; in this process, eligible workers are dynamically allocated to the selected station during scheduling rather than being fixed to stations in advance.
- We design an APAL-specific tripartite heterogeneous graph representation that jointly encodes operation precedence, station compatibility, worker skills, and physical station-worker relations.
- We develop an autoregressive pointer-based PPO policy that sequentially constructs feasible multi-resource assignments using adaptive masks, avoiding a monolithic flattened action representation.
- We evaluate the method on four APAL instance scales and report both makespan and runtime results, including offline training cost and online inference efficiency.

The remainder of this paper is organized as follows: Section II details the problem formulation and MDP modeling. Section III introduces the proposed HGP-PPO framework. Section IV analyzes the experimental results, and Section V concludes the paper.

II. PROBLEM FORMULATION AND MDP MODELING

A. Assumptions

We use the following standard manufacturing assumptions. Operations are non-preemptive; transfer and setup times are included in standard processing times; workers and stations are continuously available without breakdowns or maintenance during the planning horizon; and required materials and components are sufficiently supplied at their stations.

TABLE II
MATHEMATICAL NOTATIONS AND DECISION VARIABLES

Category	Notation	Physical Meaning
Sets	$\mathcal{O}$	Set of Operations $\{1, 2, \ldots, N\}$
	$\mathcal{S}$	Set of Stations $\{1, 2, \ldots, M\}$
	$\mathcal{W}$	Set of Heterogeneous Workers $\{1, 2, \ldots, K\}$
Parameters	d_i^{std}	Standard processing time of operation i
	q_i	Required worker demand for operation i
	sk_i	Required skill for operation i
	η_k	Personal efficiency factor of worker k
	λ	Synergistic discount factor for multi-worker collaboration
	$\mathbf{S}_{k,sk_i}$	1 if worker k masters skill sk_i, 0 otherwise
	Loc_k	Initial station binding of worker k (0: unbound)
	$slot_m$	Physical slot capacity of station m
Decisions	$x_{i,m} \in \{0, 1\}$	1 if operation i is assigned to station m
	$y_{i,k} \in \{0, 1\}$	1 if worker k is assigned to operation i
	$b_{k,m} \in \{0, 1\}$	1 if worker k is bound to station m in the schedule

B. Mathematical Formulation of the Scheduling Problem

The multi-resource pulse assembly line scheduling problem can be mathematically formulated as follows. Let $\mathcal{O} = \{1, 2, \ldots, N\}$ denote the set of operations, $\mathcal{S} = \{1, 2, \ldots, M\}$ denote the set of physically isolated stations, and $\mathcal{W} = \{1, 2, \ldots, K\}$ denote the set of available heterogeneous workers. The topological precedence between operations is defined by a directed acyclic graph, where $(i, j) \in \mathcal{E}$ indicates that operation i must strictly precede operation j.

For each operation $i \in \mathcal{O}$, d_i^{std} is the standard duration, q_i is the required worker demand, sk_i is the required skill, and $A_{i,m} \in \{0,1\}$ indicates whether station m is feasible. For each worker $k \in \mathcal{W}$, η_k represents the personal efficiency factor, $S_{k,sk_i} \in \{0,1\}$ indicates the capability to perform skill sk_i, and $Loc_k \in \{0,1,\ldots,M\}$ specifies the initial station binding (0 indicates an unbound worker). For each station $m \in \mathcal{S}$, $slot_m$ denotes concurrent operational capacity (i.e., maximum physical slots).

The binary variables $x_{i,m}$, $y_{i,k}$, and $b_{k,m}$ respectively assign operations to stations, workers to operations, and workers to at most one station. Let B_i and C_i be the start and completion times of operation i. The occupancy indicators $x_{i,m}^{(\tau)}$ and $y_{i,k}^{(\tau)}$ indicate whether operation i occupies station m and worker k at physical time τ.

The objective is to minimize the overall makespan C_{max}:

$$\min C_{max} = \max_{i \in \mathcal{O}} C_i \tag{1}$$

Remark: In a 5-station APAL, the makespan is proportional to the physical pulse cycle time ($C_{max} = 5 \times$ Cycle Time). Thus, C_{max} is used as the optimization objective for reducing the production pulse cycle.

Subject to the following constraints:

$$C_i = B_i + \frac{d_i^{std} \times q_i}{\left[\left(\sum_{k \in \mathcal{W}} \eta_k \times y_{i,k}\right) \times \lambda^{(q_i - 1)}\right]}, \quad \forall i \in \mathcal{O} \tag{2}$$

$$B_j \geq C_i, \quad \forall (i,j) \in \mathcal{E} \tag{3}$$

$$\sum_{m \in \mathcal{S}} x_{i,m} = 1, \quad x_{i,m} \leq A_{i,m}, \quad \forall i \in \mathcal{O}, \forall m \in \mathcal{S} \tag{4}$$

$$\sum_{m \in \mathcal{S}} m \times x_{j,m} \geq \sum_{m \in \mathcal{S}} m \times x_{i,m}, \quad \forall (i,j) \in \mathcal{E} \tag{5}$$

$$\sum_{k \in \mathcal{W}} y_{i,k} = q_i, \quad y_{i,k} \leq S_{k,sk_i}, \quad \forall i \in \mathcal{O}, \forall k \in \mathcal{W} \tag{6}$$

$$x_{i,m} + y_{i,k} \leq 1 + b_{k,m}, \quad \forall i, m, k, \tag{7}$$

$$\sum_{m \in \mathcal{S}} b_{k,m} \leq 1, \qquad \forall k,$$

$$b_{k,Loc_k} = 1, \qquad \forall k : Loc_k \neq 0.$$

$$\sum_{i \in \mathcal{O}} y_{i,k}^{(\tau)} \leq 1, \quad \sum_{i \in \mathcal{O}} x_{i,m}^{(\tau)} \leq slot_m, \quad \forall k, m, \tau \tag{8}$$

Constraint (2) calculates the non-linear actual processing time, incorporating worker efficiency and the synergistic discount factor $\lambda \in (0,1]$. A smaller λ represents a stronger coordination loss when more workers collaborate on the same operation. Constraint (3) enforces temporal precedence. Constraint (4) assigns each operation to exactly one feasible station. Constraint (5) guarantees the unique spatial monotonicity of pulse lines. Constraint (6) ensures the multi-worker demand and skill matching. Constraint (7) linearly links (4) and (6): assigning worker k at station m sets $b_{k,m} = 1$, preventing later cross-station use. Inference enforces this through masks and first-use binding, without solving the formulation. Constraint (8) enforces worker exclusivity and station slot capacity.

C. Markov Decision Process (MDP) Modeling

To solve the aforementioned NP-hard problem via DRL, we transform it into an event-driven MDP sequence, where t is the decision step corresponding to physical time τ_t.

1) State Space $\mathcal{S}_{MDP}$: The state s_t is encoded as a dynamic heterogeneous graph. Operation features include scheduling/readiness status, normalized processing time, required skill, and worker demand, while precedence and fixed-station feasibility are tracked by masks and constraint updates. Worker features include efficiency, skill capability, projected waiting time, free/busy state, and current station binding. Station features include accumulated load, available slot ratio, slot-release waiting time, bound/mobile worker availability, load share, and relative load competition.

2) Action Space $\mathcal{A}$ and Masking: An action a_t assigns an operation, station, and worker team. The dynamic mask $M(s_t)$ removes operations with unfinished predecessors; stations violating fixed-station, spatial-flow, capacity, or eligible-worker-count rules; and workers lacking skill or availability, excluded by the queue, already selected, or bound to another station. Unbound workers remain eligible.

3) Event-Driven State Transition $\mathcal{P}$: Given $(O_{act}, S_{act}, \mathcal{W}_{team})$, the simulator sets the earliest feasible start time from the current time, predecessor completion, worker release, and station-slot availability. It checks station capacity over $[B_i, C_i)$, binds previously unbound selected workers permanently to S_{act}, reserves them until C_i, releases resources at completion events, and advances τ to the nearest future completion event, yielding one assignment decision for each operation.

4) Dense Reward Function $\mathcal{R}$: The immediate reward r_t is designed to alleviate credit assignment challenges for extremely long sequences:

$$r_t = -(C_{max_est}^{(t)} - C_{max_est}^{(t-1)}) \tag{9}$$

where C_{max_est} denotes the estimated makespan, which is dynamically computed at each step t as:

$$C_{max_est}^{(t)} = \max\left(\max_{i \in \mathcal{O}_{sched}} C_i, \max_{j \in \mathcal{O}_{unsched}} (\tau_t + d_j^{std})\right) \tag{10}$$

incorporating both current progress (i.e., actual completion times of scheduled tasks) and heuristic lower bounds for unassigned operations. By penalizing incremental growth in projected completion time, the reward encourages shorter feasible schedules and reduces sparse-reward credit assignment difficulty. Invalid or deadlocked decisions are excluded by masks whenever possible and are not treated as feasible schedules during evaluation.

III. Proposed Method: HGP-PPO

A. Overall Architecture

To reduce the flattened action-space burden in multi-resource scheduling, we propose the Heterogeneous Graph Pointer-Proximal Policy Optimization (HGP-PPO) framework.

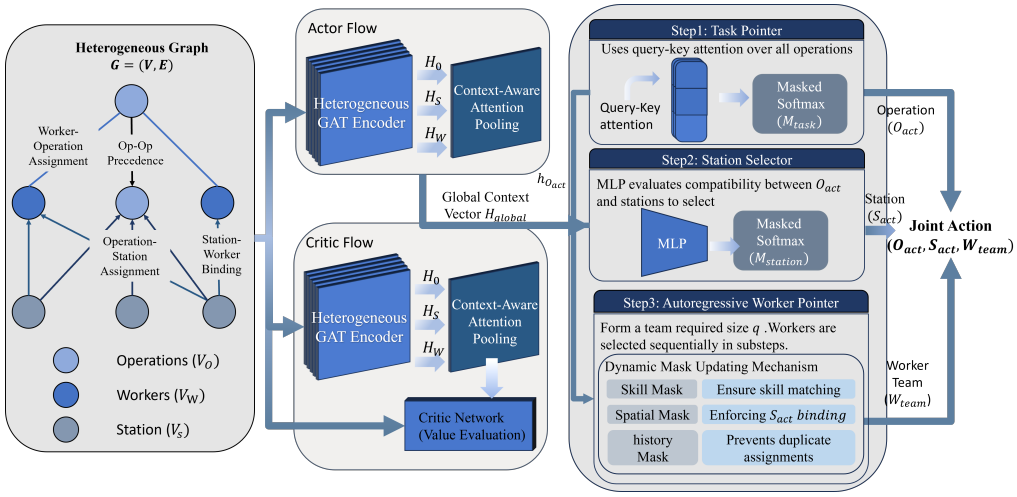


Fig. 1. Overall architecture of HGP-PPO. The APAL state is modeled as a tripartite graph and processed by a dual-stream HeteroGAT; the Actor uses masked autoregressive pointers for operation, station, and variable-length worker-team selection, while an isolated Critic stabilizes value estimation.

The architecture consists of three core modules: 1) a Tripartite Heterogeneous Graph Encoder to extract structural state representations; 2) an Attention-Based Global Context Pooling layer; and 3) an Autoregressive Pointer Network Decoder to sequentially output feasible joint actions.

B. Tripartite Heterogeneous Graph Construction

At each decision step t, the environment state s_t is mapped into a directed heterogeneous graph $G = (V, E)$. The core node set $V = V_O \cup V_S \cup V_W$ comprises three disjoint subsets: operation nodes (V_O), station nodes (V_S), and worker nodes (V_W). The reported implementation uses 18-dimensional operation features, 15-dimensional station features, and 22-dimensional worker features, while the APAL decision space remains the operation-station-worker assignment.

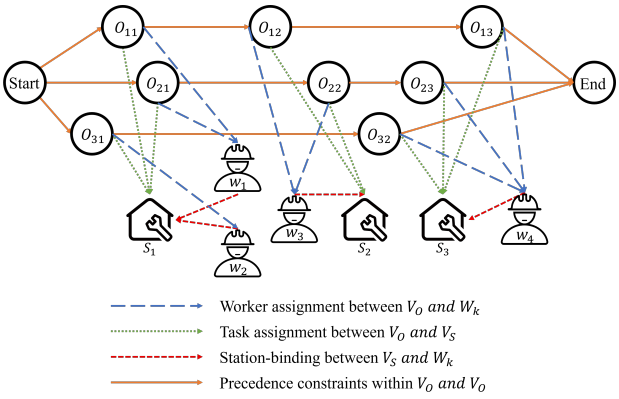


Fig. 2. Illustration of the APAL heterogeneous graph, defining the distinct resource nodes and relation-specific scheduling edges.

The edge set E captures structural and physical correlations across the tripartite graph: operation-operation edges encode precedence, worker-operation edges encode skill capability and requirements, operation-station edges encode spatial compatibility and dynamic assignment status, operation-worker edges record assigned workers for scheduled operations, and station/worker binding features represent rigid station affiliations and mobile-worker availability.

C. Heterogeneous Graph Attention Encoder

To capture the complex resource competition and temporal dependencies, we employ a multi-layer Heterogeneous Graph Attention Network (GAT). For a target node i and its neighbor j under edge relation $r \in \mathcal{R}$, the attention coefficient $e_{ij,r}$ at the l-th layer is computed as:

$$e_{ij,r} = a_r^T \times \text{LeakyReLU}\Big(W_{r,dst} \times h_i^{(l-1)} + W_{r,src} \times h_j^{(l-1)}\Big) \tag{11}$$

where $W_{r,dst}$ and $W_{r,src}$ are relation-specific weight matrices, and a_r is the attention vector. The normalized attention weight $\alpha_{ij,r}$ is obtained via the Softmax function over the neighborhood $\mathcal{N}_i^r$. The node embedding is then updated by aggregating messages from all relations with a residual connection to alleviate over-smoothing.

D. Bottleneck-Aware Context-Aware Attention Pooling

To provide a macroscopic view for the Actor-Critic networks, we compress the node-level embeddings into a fixed-length global context vector H_{global}. Instead of simple mean pooling, we employ a **Context-Aware Attention Pooling** mechanism for all three node types to dynamically capture the structural importance of bottleneck stations or critical operations:

$$H_{global} = \big[\text{Pool}_{ctx}(H_O) \parallel \text{Pool}_{ctx}(H_S) \parallel \text{Pool}_{ctx}(H_W)\big] \tag{12}$$

where Pool_{ctx} utilizes a learned attention network to compute normalized context weights for node aggregation. This helps stabilize the value evaluation in the Critic network by adaptively highlighting the most constrained resources across the heterogeneous graph.

E. Autoregressive Pointer Network Decoder

To effectively mitigate the combinatorial explosion of the joint action space, we design a variable-length autoregressive decoder. It factorizes the joint action probability into a chain of conditional probabilities:

Step 1: Task Pointer. We employ a query-key attention mechanism to score operations based on the global context vector H_{global}. Subsequently, the selection probability distribution over all operations is obtained via a masked Softmax function.

Step 2: Station Selector. Given the selected operation O_{act}, a Multi-Layer Perceptron (MLP) evaluates the compatibility with each station to output $P(S_{act})$.

Step 3: Autoregressive Worker Pointer. To construct a worker team of required size q, the decoder decomposes team selection into q conditional pointer steps. At substep r, the probability of selecting worker w_r is conditioned on the selected operation O_{act}, the selected station S_{act}, and the previously selected workers $\{w_1, \ldots, w_{r-1}\}$. After each selection, the partial team and worker mask are updated before the next pointer step. The mask removes workers that violate skill, station-binding, availability, or duplicate-selection constraints.

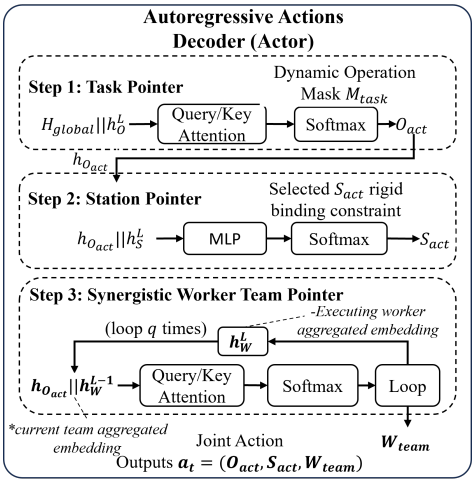


Fig. 3. Autoregressive pointer decoding with sequential factorization and dynamic mask updating.

F. Dual-Stream Critic Isolation for Stable PPO

To prevent the high-variance value gradients of long-horizon scheduling tasks from destabilizing the underlying graph representations, we physically isolate the Actor and Critic networks. The Critic utilizes an independent heterogeneous graph encoder (θ_{critic_enc}) and attention pooling layer, ensuring stable value estimation without corrupting the Actor's topological feature extraction.

IV. Experiments and Results Analysis

A. Experimental Setup

Each APAL instance is a scheduling graph with precedence, processing times, skill and worker-team demands, fixed-station restrictions, station capacities, worker skills, efficiencies, and station-binding constraints. The training pool contains 50 randomized 400–800-operation instances generated from real manufacturing data by topology-preserving deletion and augmentation with deadlock avoidance, followed by randomization of processing times, skill types, and worker-team demands. Testing uses four deterministic real-factory benchmarks: Medium (283 operations), Large (680 operations), Industrial-I (2338 operations), and Industrial-II (3182 operations), evaluating cross-instance application under the same constraint schema.

For learning-based methods and architectural ablations, we report the mean and standard deviation over ten independent training seeds. For dispatching rules and search-based methods, we report fixed-budget deterministic or single-run results under the same APAL feasibility checker and benchmark instances.

The PyTorch implementation uses a PPO learning rate of 5×10^{-5} with Actor/Critic multipliers of 0.5/1.0, $\gamma = 0.999$, GAE parameter 0.9, two PPO epochs per rollout, and clip annealing from $\epsilon = 0.2$ to 0.10. The reported training batch size is 256. The graph hidden dimension is 128, with 4 GAT heads and 5 graph attention layers; deterministic evaluation uses temperature 0. Runtime is reported as end-to-end validation wall time and internal inference time, the latter isolating scheduling computation.

Baselines include dispatching rules (SPT, LPT, Random, EDD, CPM, and MSL), search methods (Beam Search, Iterated Greedy, and Simulated Annealing), flattened DQN/PPO, and HGP-PPO ablations (w/o GAT and w/o Attn. Pool). All use the same APAL feasibility checker and makespan objective. Beam Search uses width 4, 4 expansions per candidate, depth 8, and 4-level early stopping; IG uses 80 destroy-and-repair iterations with 10% destruction and 0.25 repair noise; SA uses 120 iterations, initial temperature 0.05, cooling rate

Algorithm 1 Training Procedure of HGP-PPO

Require: APAL training pool, rollout horizon N, PPO epochs K, clip range ϵ
Initialize Actor encoder and pointer policy θ_π; initialize isolated Critic stream θ_V
for each training update **do**
  Sample an APAL instance and construct the initial tripartite graph state s_0
  for $t = 0$ **to** $N - 1$ **do**
    Encode operations, stations, and workers; aggregate node embeddings by attention pooling
    Select O_{act} and S_{act} under operation, station, capacity, and binding masks
    Initialize $\mathcal{W}_{team} \leftarrow \emptyset$ and dynamic worker mask $M_{worker}^{(1)}$
    for $r = 1$ **to** $q_{O_{act}}$ **do**
      Select w_r conditioned on $(O_{act}, S_{act}, \mathcal{W}_{team})$ and refresh $M_{worker}^{(r+1)}$
    end for
    Execute $a_t = (O_{act}, S_{act}, \mathcal{W}_{team})$, update station bindings, and store transition, reward, and log probability
  end for
  Estimate advantages and returns with GAE
  for $k = 1$ **to** K **do**
    Update θ_π with the clipped PPO objective and θ_V with value loss
  end for
end for

TABLE III

MAKESPAN AND ONLINE RUNTIME COMPARISON ACROSS APAL SCALES. RUNTIME IS REPORTED IN SECONDS UNDER THE REPORTED SEARCH BUDGETS; FOR LEARNED POLICIES IT DENOTES INTERNAL INFERENCE TIME, AND FOR DISPATCHING/SEARCH METHODS IT DENOTES SCHEDULING OR SEARCH TIME. LOWER IS BETTER.

Method	**283 Ops**		**680 Ops**		**2338 Ops**		**3182 Ops**	
	Makespan	**Runtime**	**Makespan**	**Runtime**	**Makespan**	**Runtime**	**Makespan**	**Runtime**
SPT	178.17	1.11	498.37	3.23	1339.75	24.33	1796.69	35.66
LPT	146.21	1.11	524.87	3.23	1214.62	25.77	1692.17	37.18
Random	162.19	1.13	556.71	3.26	1420.30	23.84	1942.50	35.05
EDD	161.51	1.12	514.13	3.55	1322.72	25.34	1877.63	36.62
CPM	148.11	1.12	476.68	3.96	1168.85	35.56	1691.39	48.90
MSL	178.17	1.10	498.37	3.31	1339.75	24.50	1796.69	35.53
Beam Search	147.35	83.37	454.55	399.92	1074.00	2961.02	1513.92	3784.29
IG	141.42	94.35	494.70	193.99	1187.98	2584.85	1557.41	4123.31
SA	162.52	147.97	451.83	605.03	1070.25	4775.06	1396.80	8330.25
DQN	160.64±11.12	1.31	543.78±31.61	3.60	1289.11±40.58	25.22	1739.35±51.89	36.29
PPO	214.49±6.74	1.34	704.80±30.60	3.42	1839.50±29.15	25.26	2572.75±21.68	37.19
HGP-PPO	**140.51±7.32**	10.37	**306.33±18.10**	27.80	**703.44±33.76**	160.56	**1051.74±25.27**	168.26
w/o GAT	151.15±7.36	2.32	361.21±25.19	5.70	954.78±77.93	31.53	1394.09±87.90	45.25
w/o Attn. Pool	182.22±8.44	7.53	567.95±41.34	18.04	1617.52±63.84	73.95	2149.67±230.97	103.79

0.96, and minimum temperature 10^{-4}.

B. Comparative Analysis on Makespan and Efficiency

Table III reports makespan and online scheduling runtime across four APAL scales. Under fixed baseline budgets, HGP-PPO achieves the best average makespan among learning-based methods and remains competitive with tested dispatching/search baselines on large cases. Compared with flattened PPO, it reduces makespan from 1839.50 to 703.44 on 2338 operations and from 2572.75 to 1051.74 on 3182 operations. Its 15.587 h training cost is offline and separated from per-instance online scheduling time.

C. Ablation Study

Table III also reports two variants: **w/o GAT**, which removes the heterogeneous graph encoder, and **w/o Attn. Pool**, which removes attention-based global pooling. Both worsen makespan across all scales, indicating that graph-structured resource encoding and global context aggregation are useful for APAL scheduling. Ablation runtimes are from deterministic validation, while makespan values follow the repeated-seed protocol.

V. CONCLUSION AND FUTURE WORK

This paper presented HGP-PPO, a graph-based reinforcement learning framework for multi-resource APAL scheduling. By combining a tripartite heterogeneous graph with an autoregressive pointer decoder, HGP-PPO factorizes coupled operation-station-worker decisions and embeds major feasibility constraints into sequential decoding. Results show improvements over flattened PPO/DQN and competitive performance against fixed-budget dispatching and search methods, especially on large APAL instances. Future work will address stochastic disturbances such as machine breakdowns or worker absences, and compare HGP-PPO with emerging graph-transformer schedulers.

REFERENCES

[1] M. E. Salveson, "The assembly-line balancing problem," *Transactions of the American Society of Mechanical Engineers*, vol. 77, no. 6, pp. 939–947, 1955.

[2] M. Panzer and B. Bender, "Deep reinforcement learning in production systems: a systematic literature review," *International Journal of Production Research*, vol. 60, no. 13, pp. 4316–4341, 2022.

[3] W. Song *et al.*, "Flexible job-shop scheduling via graph neural network and deep reinforcement learning," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1600–1610, 2023.

[4] C. L. Liu and T. H. Huang, "Dynamic job-shop scheduling problems using graph neural network and deep reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 11, pp. 6836–6848, 2023.

[5] J. P. Huang, L. Gao, and X. Y. Li, "An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem," *Expert Systems with Applications*, vol. 238, p. 121756, 2024.

[6] Y. Zhao, Z. Zhu, C. Gao, *et al.*, "Heterogeneous graph reinforcement learning for dependency-aware multi-task allocation in spatial crowdsourcing," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3417–3434, 2026.

[7] Y. Hu *et al.*, "Dynamic scheduling for dual-resource constrained flexible job-shop via semantic-aware graph modelling and deep reinforcement learning," *Knowledge-Based Systems*, vol. 328, p. 114192, 2025.

[8] R. Zhong *et al.*, "Solving flexible job-shop problem considering skilled workers via multi-domain graph attention network," *International Journal of Production Research*, vol. 64, no. 13, pp. 5541–5562, 2026.

[9] Z. Yi *et al.*, "An end-to-end deep reinforcement learning method for dynamic flexible job shop scheduling problem considering multi-skilled worker constraints," *Journal of Intelligent Manufacturing*, early access, 2026, doi: 10.1007/s10845-025-02784-1.

[10] E. Yuan *et al.*, "Solving flexible job shop scheduling problems via deep reinforcement learning," *Expert Systems with Applications*, vol. 245, p. 123019, 2024.

[11] J. Ai, F. Qiao, J. Liu, *et al.*, "A graph-enhanced deep-reinforcement learning approach for the aircraft pulse assembly line balancing problem," in *Proc. 2025 China Automation Congress (CAC)*, 2025, pp. 6623–6628.